



Seminario de Programación



C++ - Biblioteca Qt



Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

1) Introducción a Qt

1.1 ¿Qué es Qt?

Qt es un framework multiplataforma para el desarrollo de aplicaciones con interfaz gráfica y componentes de software en **C++**. Permite crear programas visuales que funcionan en **Linux, Windows, macOS** y otros sistemas sin modificar el código principal.

Qt ofrece:

- Una arquitectura modular con bibliotecas para **interfaz gráfica, redes, bases de datos y multimedia**.
- Un sistema propio de **signals y slots** que simplifica la comunicación entre objetos.
- Herramientas de desarrollo integradas como **Qt Creator**, que facilitan el diseño y compilación de proyectos.

En este curso utilizaremos **Qt6**, la versión más reciente y compatible con **CMake**, el estándar moderno para la compilación de proyectos C++.

1.2 Casos de uso y módulos principales

Qt se utiliza ampliamente en distintos tipos de aplicaciones:

- **Escritorio:** herramientas de productividad, editores, configuradores.
- **Integración industrial:** paneles de control, instrumentación.
- **Aplicaciones científicas y educativas:** visualización de datos y simulaciones.
- **Aplicaciones móviles o embebidas:** interfaces livianas y reactivas.

Módulos principales

- **QtCore:** clases base, manejo de señales, temporizadores, archivos, hilos.
- **QtGui:** dibujo, fuentes, imágenes y eventos de teclado o ratón.
- **QtWidgets:** controles de interfaz tradicionales (botones, formularios, menús).
- **QtCharts:** creación de gráficos interactivos (líneas, barras, torta).
- **QtQuick / QML:** alternativa declarativa moderna para interfaces, no cubierta en este curso.

En este curso se trabajará con **Qt Widgets**, ya que constituye la forma más directa de crear interfaces gráficas utilizando únicamente **C++**. Su enfoque permite comprender los fundamentos del modelo de objetos, eventos y señales de **Qt**, sin introducir nuevos lenguajes ni capas adicionales. **Qt Quick / QML**, en cambio, ofrece un enfoque más visual y declarativo, ideal para interfaces modernas, animadas o adaptadas a dispositivos móviles, pero se recomienda explorarlo una vez dominados los conceptos básicos de **Qt Widgets**.

2) Instalación y configuración (Linux: Debian/Ubuntu)

2.1 Requisitos previos

Antes de comenzar, asegúrate de contar con:

- Un sistema **Debian, Ubuntu** o derivado actualizado.
- Herramientas de compilación de C++ (`g++` , `make`).
- Conexión a Internet para instalar dependencias y ejecutar el instalador de Qt.

2.2 Instalación de Qt 6 (Open Source) y CMake

En este curso se utilizará la **versión Open Source de Qt 6**, disponible en:

<https://www.qt.io/download-open-source>

Esta versión se distribuye bajo licencia **LGPL**, lo que permite su uso libre para fines **académicos, educativos y de investigación**. Se recomienda instalarla mediante el **Qt Online Installer**, que incluye los módulos principales (`Core` , `Widgets` , `Charts`) y el entorno **Qt Creator**.

1. Preparar entorno base

Instala las herramientas de compilación necesarias:

```
sudo apt update
sudo apt install -y build-essential cmake
```

Estas herramientas permiten compilar proyectos Qt mediante **CMake** y **make** en Linux.

Nota: si aparece el error *"Could not load platform plugin 'xcb'"*, instala la biblioteca requerida:

```
sudo apt install -y libxcb-xinerama0
```

2. Instalar Qt 6 (Open Source) con el Online Installer

1. Descarga el instalador desde la página oficial: <https://www.qt.io/download-open-source> (archivo `.run` , por ejemplo `qt-online-installer-linux-x64-<version>.run`)

2. Hazlo ejecutable y ejecútalo desde la terminal:

```
chmod +x qt-online-installer-linux-x64-<version>.run
./qt-online-installer-linux-x64-<version>.run
```

3. Durante la instalación:

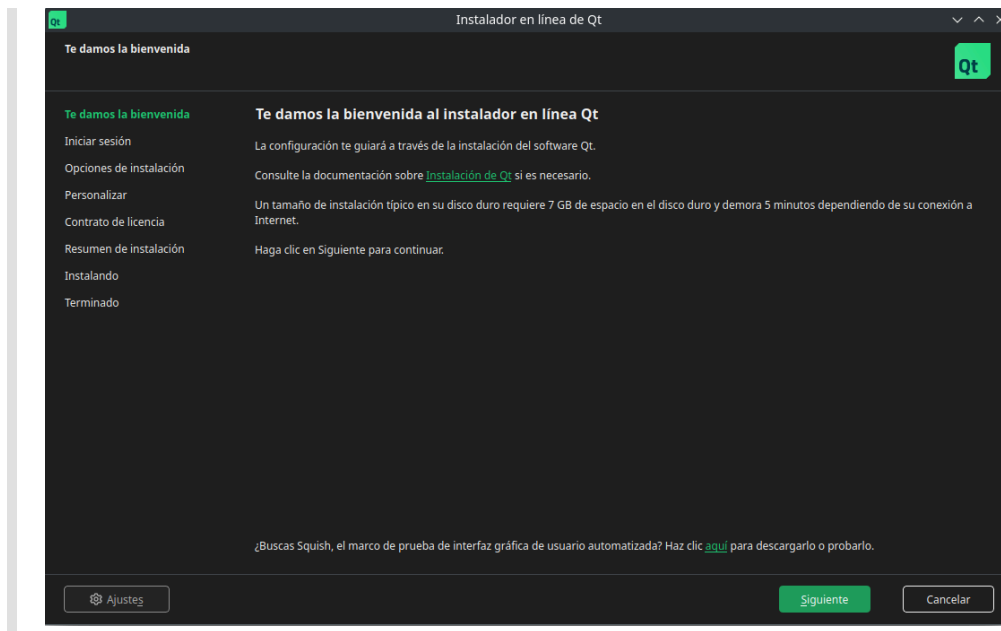
- **Inicia sesión o crea una cuenta gratuita de Qt.** Es requisito del instalador y permite acceder a la versión **Open Source (LGPL)**. Registro disponible en: <https://login.qt.io/register>
- Selecciona **Qt 6 for Desktop Development**.
- Incluye los módulos **Qt Charts** y **Qt Tools**.

4. Proceso de instalación

A continuación se muestran las principales pantallas del instalador:

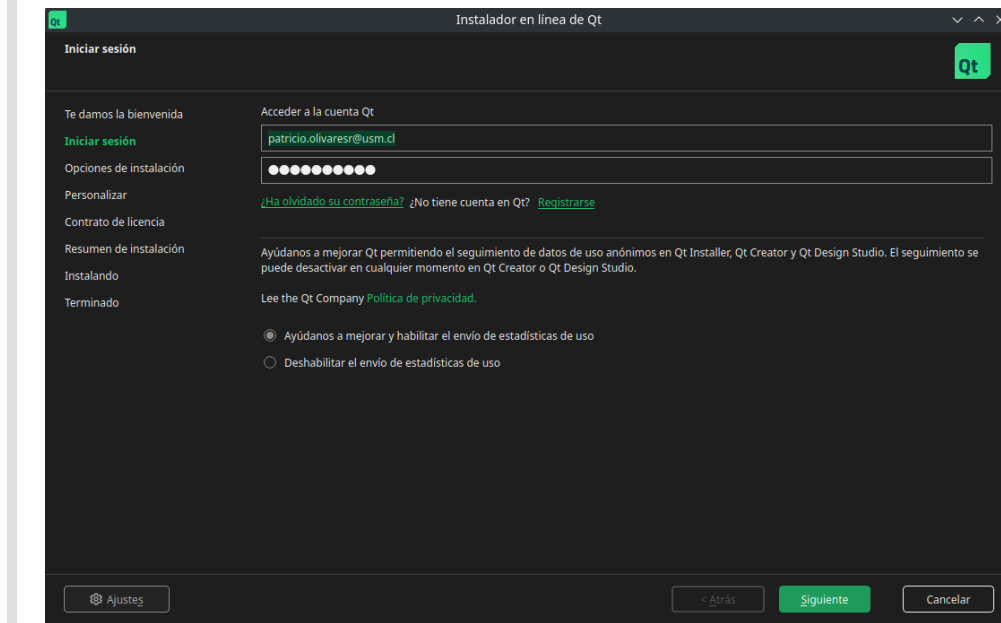
Pantalla 1 Bienvenida

Presenta la introducción al proceso de instalación. Presiona **Siguiente** para continuar.



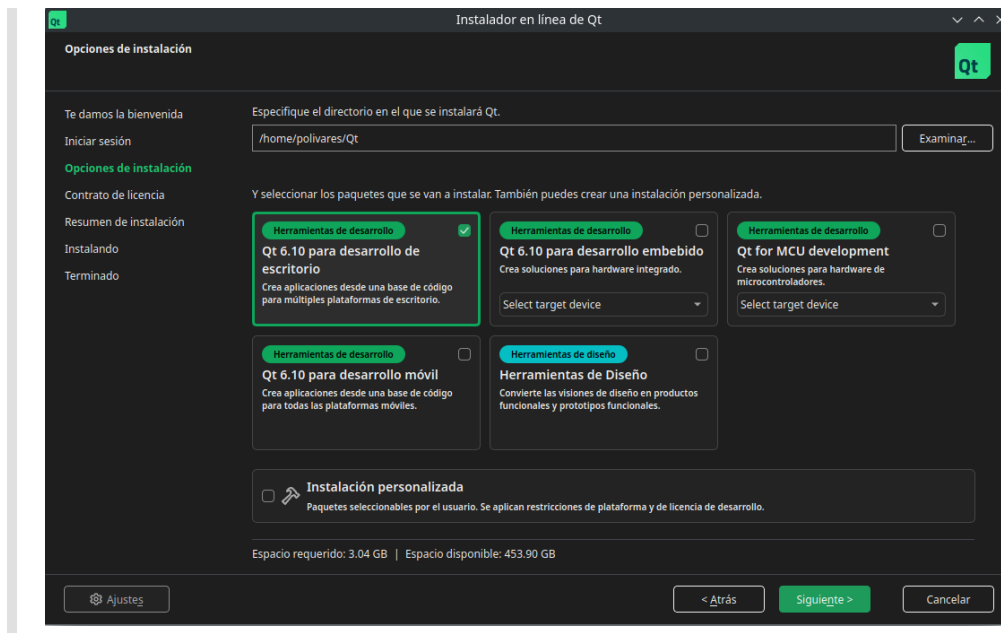
Pantalla 2 Inicio de sesión

Ingresa tu correo y contraseña de la cuenta Qt o regístrate si aún no la tienes.



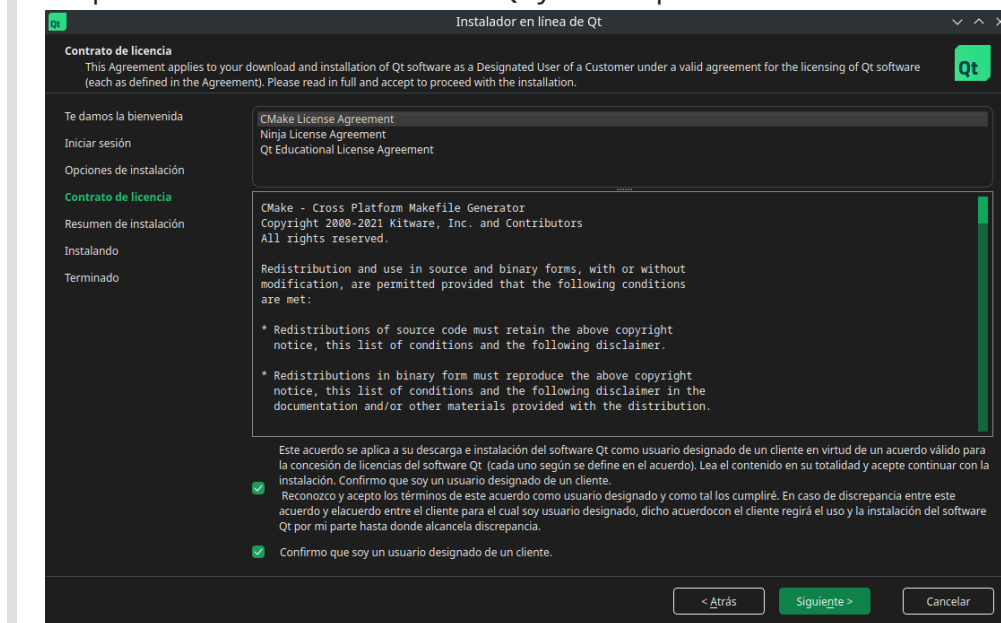
Pantalla 3 Opciones de instalación

Elige el directorio de instalación (por defecto `/home/usuario/Qt`) y selecciona **Qt 6 for desktop development**.



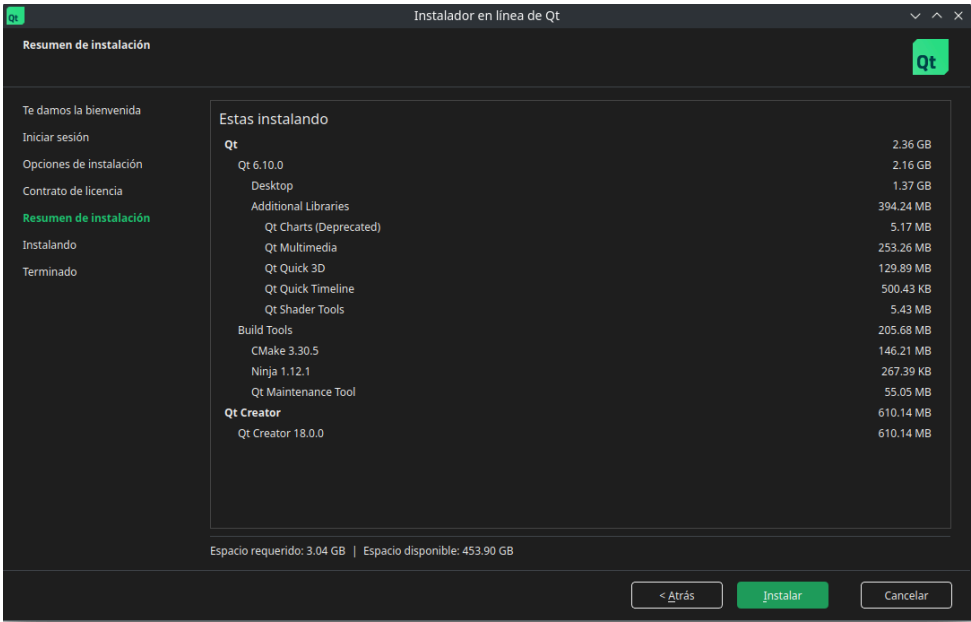
Pantalla 4 Contrato de licencia

Acepta los términos de licencia de Qt y CMake para continuar.



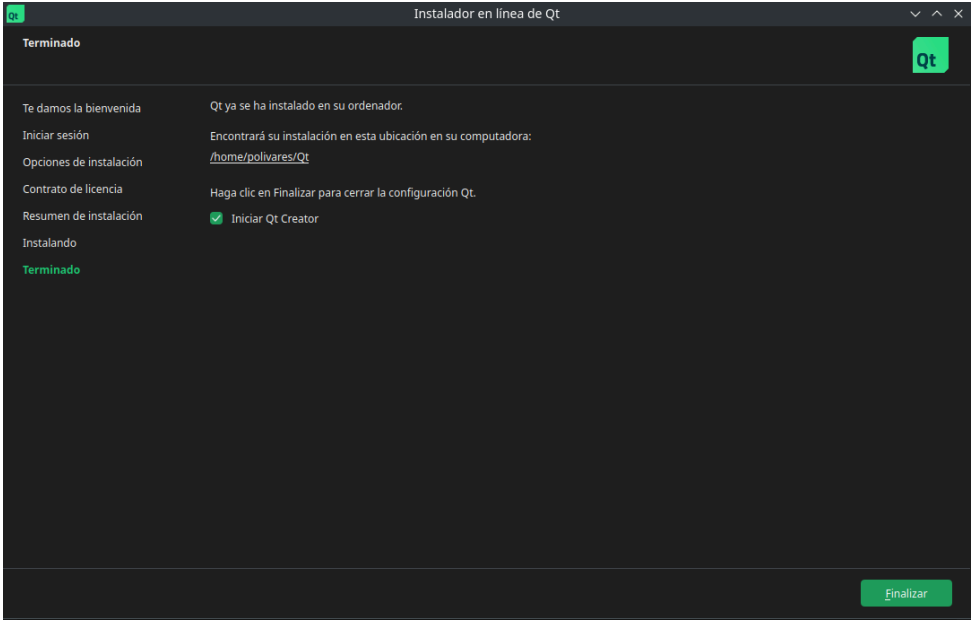
Pantalla 5 Resumen de instalación

Revisa los componentes seleccionados, asegurando que estén marcados **Qt Charts**, **Qt Tools** y **Qt Creator**.



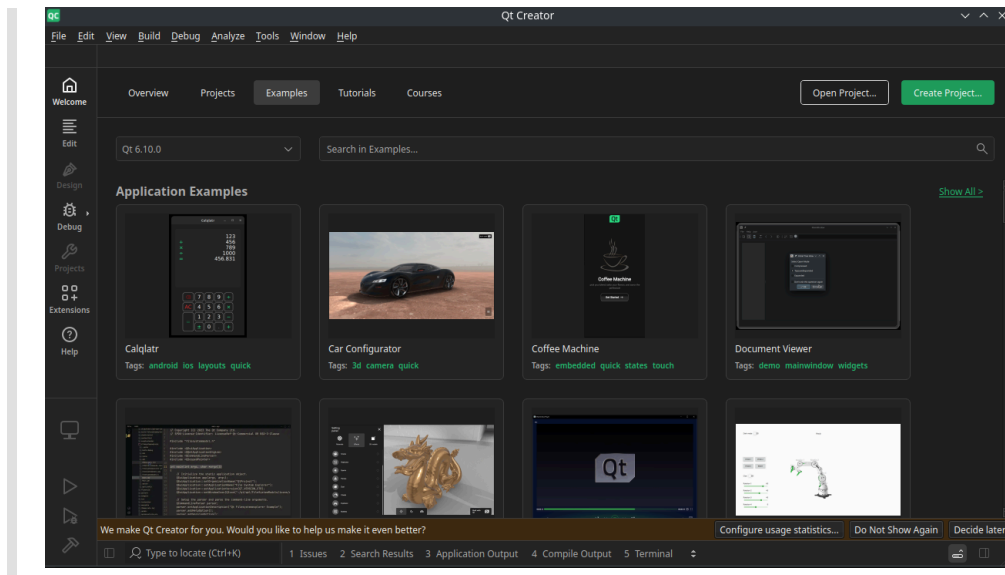
Pantalla 6 Progreso e instalación finalizada

Espera a que el proceso termine. Puedes marcar la opción **Iniciar Qt Creator** al finalizar.



Pantalla 7 Qt Creator iniciado

Al finalizar, se abrirá **Qt Creator** mostrando ejemplos y plantillas listas para comenzar un nuevo proyecto.



3. Verificación

Comprueba que Qt y CMake estén disponibles:

```
cmake --version
qmake6 --version
```

Si ambos comandos muestran versiones recientes, el entorno está correctamente configurado para compilar proyectos **Qt 6 (Open Source)** con **CMake** y **make**.

3) Estructura de proyecto con CMake (Qt6)

3.1 Estructura de archivos y carpetas

Base creada desde **Qt Creator** → *New Project* → *Qt Widgets Application*.

```
mi-app/
├── CMakeLists.txt
├── main.cpp
├── mainwindow.h
├── mainwindow.cpp
├── mainwindow.ui
└── build/
```

- **CMakeLists.txt:** Archivo de configuración que define cómo se compila el proyecto. Indica las fuentes, dependencias y módulos de Qt que deben incluirse.
- **main.cpp:** Punto de entrada del programa. Crea el objeto `QApplication`, inicializa la ventana principal y ejecuta el bucle principal de eventos.

- **mainwindow.h:** Define la clase principal de la interfaz (MainWindow), sus métodos y señales. Es el encabezado que declara la estructura de la ventana.
- **mainwindow.cpp:** Implementa la lógica de la interfaz y el comportamiento de los widgets definidos en `mainwindow.h` y `mainwindow.ui`.
- **mainwindow.ui:** Archivo XML generado por el diseñador gráfico de Qt Creator (**Qt Designer**). Contiene la disposición visual de la interfaz (botones, menús, layouts, etc.).
- **build/** Carpeta donde CMake genera todos los archivos de compilación (Makefiles, binarios, temporales). Se recomienda mantenerla separada del código fuente.

Notas rápidas (Qt Creator):

- El **Kit** seleccionado define compilador + Qt + generador (Makefiles).
- El `.ui` se edita con el **Designer** integrado.
- El directorio **build/** lo maneja Qt Creator; puede ponerse fuera del proyecto.

3.2 CMakeLists.txt base

El archivo **CMakeLists.txt** define la estructura del proyecto y las dependencias que se deben incluir durante la compilación. En proyectos Qt, permite agregar **archivos fuente** y **módulos** (como `Widgets`, `Charts`, etc.) de forma centralizada.

Ejemplo (generado por Qt Creator):

```
cmake_minimum_required(VERSION 3.19)
project(test1 LANGUAGES CXX)

find_package(Qt6 6.5 REQUIRED COMPONENTS Core Widgets)

qt_add_executable(test1
    main.cpp
    mainwindow.cpp
    mainwindow.h
    mainwindow.ui
)

target_link_libraries(test1
    PRIVATE
        Qt::Core
        Qt::Widgets
)
```

Puntos clave:

- `find_package(...)` : define los **módulos Qt** que se usarán.
- `qt_add_executable(...)` : lista los **archivos fuente** del proyecto.

- `target_link_libraries(...)` : enlaza el ejecutable con los módulos definidos.

Agregar nuevos módulos: Para incluir otros componentes, basta con agregarlos a la línea de `find_package` . Por ejemplo, para usar **Qt Charts**:

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Charts)
target_link_libraries(test1 PRIVATE Qt::Core Qt::Widgets Qt::Charts)
```

En este curso, bastará con editar este archivo para incluir nuevos módulos o archivos conforme se amplíen los proyectos.

3.3 Compilación y ejecución

En Qt Creator (recomendado para el curso):

1. **Open Project...** → seleccionar `CMakeLists.txt` .
 2. Elegir **Kit** (Qt 6 Desktop) y confirmar.
 3. Menú **Build** → **Build Project** (o run).
 4. **Run** (Ctrl+R).
- La salida se ve en *Compile Output* y *Application Output*.

4) Elementos fundamentales de Qt

Qt está construido sobre una arquitectura orientada a objetos que permite estructurar programas en clases, conectarlas entre sí y responder a eventos de manera eficiente. El núcleo de este modelo se basa en las clases derivadas de `QObject` y en el sistema de comunicación **signals y slots**.

4.1 Clases en Qt

Las clases en Qt funcionan igual que en C++, pero añaden un sistema interno que permite comunicación entre objetos y manejo de propiedades dinámicas. Casi todas las clases de Qt heredan, directa o indirectamente, de `QObject` .

Ejemplo:

```
#include <QObject>

class Contador : public QObject {
    Q_OBJECT    // habilita el sistema metaobjeto de Qt

public:
```

```
explicit Contador(QObject *parent = nullptr);
void setValor(int v);
```

```
signals:
    void valorCambiado(int nuevoValor); // signal
```

```
private:
    int m_valor = 0;
};
```

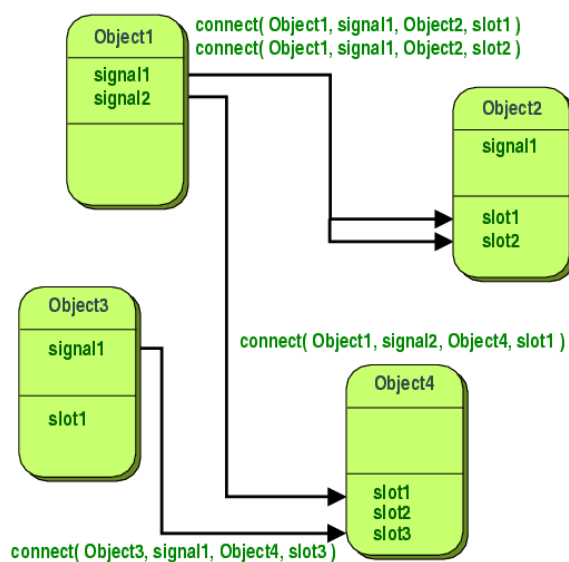
La macro `Q_OBJECT` activa el procesamiento interno de Qt (MOC), que permite usar señales, slots y otras características avanzadas.

4.2 Signals y Slots

El mecanismo **signals/slots** es el pilar del modelo de comunicación en Qt. Permite que los objetos interactúen sin depender directamente unos de otros.

- **Signal:** se emite cuando ocurre un evento.
- **Slot:** recibe el evento y ejecuta la acción correspondiente.
- **Conexión:** une ambos elementos mediante `QObject::connect`.

Ejemplo:



```
#include <QObject>
#include <QDebug>
```

```
class Emisor : public QObject {
    Q_OBJECT
signals:
    void mensajeEnviado(QString texto);
};
```

```
class Receptor : public QObject {
    Q_OBJECT
```

```
public slots:
    void mostrarMensaje(QString texto) {
        qDebug() << "Mensaje recibido:" << texto;
    }
};
```

Conexión entre objetos:

```
Emisor e;
Receptor r;
```

```
QObject::connect(&e, &Emisor::mensajeEnviado,
                &r, &Receptor::mostrarMensaje);
```

```
emit e.mensajeEnviado("Hola desde Qt!");
```

Cada vez que se emite la señal `mensajeEnviado`, el slot `mostrarMensaje` es llamado automáticamente.

4.3 Flujo general en una aplicación Qt

1. **Creación de objetos:** las clases derivadas de `QObject` gestionan jerarquías, memoria y señales.
2. **Conexión de señales y slots:** define cómo los objetos se comunican y reaccionan ante eventos.
3. **Ejecución del bucle principal:** mediante la clase `QApplication`, que mantiene el flujo de eventos activos mientras la aplicación está en ejecución.

Esquema conceptual:

```
QObject (base)
├── QCoreApplication
└── QApplication → maneja eventos y la interfaz
```

4.4 Ejemplo integrado

```
#include <QApplication>
#include <QObject>
#include <QDebug>

class Emisor : public QObject {
    Q_OBJECT
signals:
    void notificar(QString texto);
};

class Receptor : public QObject {
    Q_OBJECT
public slots:
```

```

    void recibir(QString texto) {
        qDebug() << "Recibido:" << texto;
    }
};

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    Emisor e;
    Receptor r;

    QObject::connect(&e, &Emisor::notificar, &r, &Receptor::recibir);
    emit e.notificar("Señal de prueba");

    return app.exec();
}

```

Este ejemplo muestra la estructura base de una aplicación Qt, donde los objetos se comunican mediante señales y responden con acciones definidas en sus slots.

5) Creación de un proyecto y clases en Qt Creator

5.1 Crear un nuevo proyecto Qt

1. **Abrir Qt Creator.** Al iniciar, selecciona **New Project** → **Qt** → **Qt Widgets Application**.
2. **Configurar el proyecto.**
 - Asigna un nombre (por ejemplo, `MiPrimeraApp`).
 - Elige la carpeta de destino.
 - Asegúrate de que el **Kit** seleccionado corresponda a **Qt 6 Desktop**.
3. **Seleccionar clases iniciales.** Qt Creator creará automáticamente los archivos:
 - `main.cpp`
 - `mainwindow.h`
 - `mainwindow.cpp`
 - `mainwindow.ui`
4. **Compilar y ejecutar.** Presiona **Ctrl+R** o el botón  para ejecutar el proyecto. Aparecerá una ventana básica con el título y los menús iniciales generados por Qt Creator.

5.2 Estructura generada

```

MiPrimeraApp/
├─ CMakeLists.txt
├─ main.cpp
├─ mainwindow.h
├─ mainwindow.cpp
├─ mainwindow.ui
└─ build/

```

Qt Creator organiza el proyecto automáticamente y genera un `CMakeLists.txt` similar al visto anteriormente. El archivo `.ui` se abre con **Qt Designer**, donde se puede construir la interfaz visualmente.

5.3 Crear una nueva clase

Para agregar una clase al proyecto:

1. Menú **File** → **New File or Project** → **C++** → **C++ Class**.
2. Asigna un nombre, por ejemplo: `Controlador`.
3. Qt Creator generará automáticamente:
 - `controlador.h`
 - `controlador.cpp`

Ejemplo:

```

// controlador.h
#include <QObject>

class Controlador : public QObject {
    Q_OBJECT
public:
    explicit Controlador(QObject *parent = nullptr);
    void imprimirMensaje();
};

// controlador.cpp
#include "controlador.h"
#include <QDebug>

Controlador::Controlador(QObject *parent)
    : QObject(parent) {}

void Controlador::imprimirMensaje() {
    qDebug() << "Mensaje desde la clase Controlador";
}

```

Luego, puede instanciarse desde `mainwindow.cpp`:

```

#include "mainwindow.h"
#include "ui_mainwindow.h"

```

```
#include "controlador.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    Controlador ctrl;
    ctrl.imprimirMensaje();
}
```

5.4 Edición visual con Qt Designer

1. En el panel de proyecto, haz doble clic en `mainwindow.ui`.
2. Se abrirá el editor visual (Qt Designer).
3. Arrastra widgets (botones, etiquetas, cuadros de texto) desde el panel izquierdo.
4. Guarda los cambios (`Ctrl+S`) y regresa al editor de código: Qt Creator generará automáticamente el código asociado.

Qt Creator vincula el archivo `.ui` con la clase `MainWindow` mediante `ui_mainwindow.h`, permitiendo acceder a los elementos visuales directamente desde el código.

5.5 Resultado

Al compilar nuevamente, Qt Creator construye el proyecto, genera el código necesario desde el archivo `.ui`, y ejecuta la aplicación con la interfaz visual creada.

Este flujo combina el diseño visual (Qt Designer) con la programación en C++, permitiendo ampliar fácilmente la aplicación agregando clases, señales y lógica personalizada.

6) Uso de Qt Designer y elementos gráficos

Qt Designer permite construir interfaces gráficas de manera visual dentro de **Qt Creator**, sin necesidad de escribir código para la disposición de los widgets. Esta herramienta genera automáticamente un archivo `.ui` (en formato XML) que describe la estructura y propiedades de la interfaz.

6.1 Interfaz de Qt Designer

Qt Designer se integra directamente en **Qt Creator** y se abre al hacer doble clic sobre cualquier archivo `.ui`.

Partes principales del entorno:

- **Widget Box:** panel con todos los controles disponibles (botones, etiquetas, campos de texto, menús, etc.).
- **Formulario de diseño:** área central donde se arrastran y colocan los elementos.
- **Inspector de objetos:** muestra la jerarquía de widgets en la ventana.
- **Editor de propiedades:** permite modificar textos, tamaños, colores, fuentes y comportamientos.

El archivo `.ui` se compila automáticamente en código C++ durante la construcción del proyecto, de modo que los widgets creados en el diseñador pueden utilizarse desde el código sin configuraciones adicionales.

6.2 Creación de una interfaz básica

1. Abre `mainwindow.ui` en Qt Designer.
2. Arrastra un **QPushButton** y un **QLabel** al formulario.
3. Cambia sus nombres de objeto (`objectName`) a `btnSaludo` y `lblMensaje`.
4. Guarda los cambios (`Ctrl+S`).

Desde el código, los elementos se pueden manipular a través del puntero `ui`:

```
// mainwindow.cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    ui->lblMensaje->setText("Listo para saludar");
}
```

6.3 Uso de signals y slots en Qt Designer

Qt Designer permite conectar señales y slots sin escribir código, mediante el **modo de conexión**.

Pasos para conectar visualmente:

1. Selecciona **Edit** → **Edit Signals/Slots** o presiona **F4**.

2. Haz clic en el widget emisor (por ejemplo, el botón).
3. Arrastra la flecha hasta el receptor (por ejemplo, la etiqueta o la ventana).
4. En el cuadro de diálogo, elige el **signal** del emisor y el **slot** del receptor.
 - Ejemplo: conectar `clicked()` del botón con un slot `setText()` del label.

Resultado: Qt Designer guardará esta conexión en el archivo `.ui`, y Qt Creator la generará automáticamente al compilar el proyecto.

6.4 Limitaciones del sistema visual de signals y slots

Aunque Qt Designer permite realizar conexiones simples sin código, su uso tiene algunas limitaciones:

Aspecto	Conexiones visuales (Qt Designer)	Conexiones por código (C++)
Facilidad	Muy rápida para prototipos simples.	Requiere escribir código.
Flexibilidad	Limitada a señales y slots predefinidos.	Puede conectar funciones personalizadas.
Depuración	No se muestran errores hasta tiempo de ejecución.	Errores detectados en compilación.
Condiciones lógicas	No admite condiciones o procesamiento adicional.	Total control del flujo y lógica.
Escalabilidad	Adecuado solo para interfaces pequeñas.	Escalable a proyectos grandes y modulares.

Ejemplo de conexión visual equivalente a código:

En Qt Designer:

- `QPushButton::clicked()` → `QLabel::setText("¡Hola desde Qt!")`

En código:

```
QObject::connect(ui->btnSaludo, &QPushButton::clicked, this, [this]() {
    ui->lblMensaje->setText("¡Hola desde Qt!");
});
```

En este curso se recomienda **usar Qt Designer para la disposición visual** y realizar las conexiones lógicas (signals/slots personalizados) desde el código, garantizando mayor claridad y control sobre el comportamiento de la aplicación.

7) Gráficos con Qt Charts

El módulo **Qt Charts** permite incorporar gráficos fácilmente dentro de las interfaces creadas con **Qt Widgets**. Está diseñado sobre el **Graphics View Framework**, por lo que cada gráfico puede integrarse como un widget más en la aplicación.

Tipos de gráficos disponibles:

- Líneas (`QLineSeries`)
- Barras y barras horizontales (`QBarSeries` , `QHorizontalBarSeries`)
- Áreas (`QAreaSeries`)
- Torta o dona (`QPieSeries`)
- Polares (`QPolarChart`)

Para utilizarlo, debe estar instalado el módulo `Qt Charts` y agregado al proyecto en `CMakeLists.txt`

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Charts)
target_link_libraries(miapp PRIVATE Qt::Core Qt::Widgets Qt::Charts)
```

7.1 Estructura general de un gráfico

1. Crear la **serie de datos** (por ejemplo, líneas o barras).
2. Agregar la serie a un **QChart**.
3. Insertar el gráfico dentro de un **QChartView** y mostrarlo en la ventana principal.

```
QLineSeries *series = new QLineSeries();
series->append(0, 6);
series->append(2, 4);
series->append(3, 8);

QChart *chart = new QChart();
chart->addSeries(series);
chart->createDefaultAxes();
chart->setTitle("Simple line chart example");

QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing);
setCentralWidget(chartView);
```

7.2 Gráficos de línea

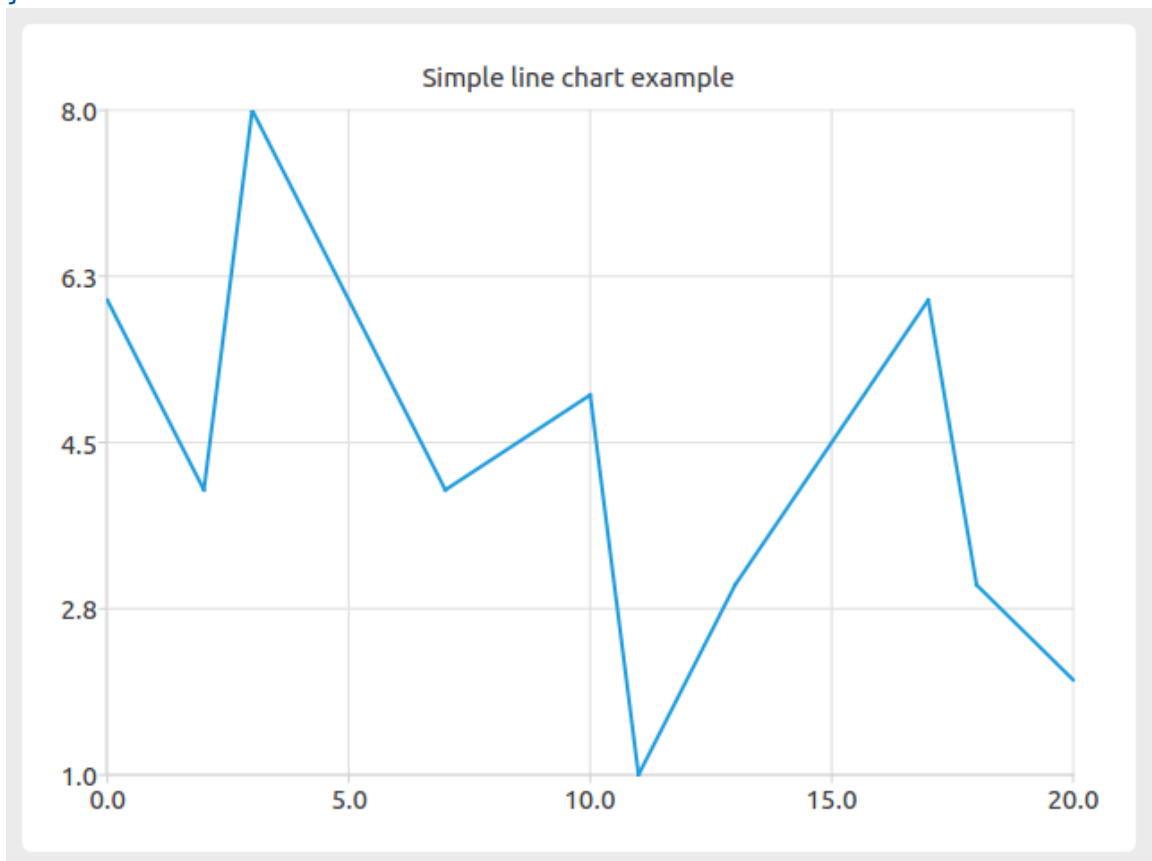
Muestran la evolución de valores a lo largo de un eje. Se utiliza la clase `QLineSeries`.

```
#include "mainwindow.h"
#include <QApplication>
```

```

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Crear serie
    QLineSeries *series = new QLineSeries();
    series->append(0, 6);
    series->append(2, 4);
    series->append(3, 8);
    series->append(7, 4);
    series->append(10, 5);
    *series << QPointF(11, 1) << QPointF(13, 3)
              << QPointF(17, 6) << QPointF(18, 3)
              << QPointF(20, 2);
    // Crear gráfico e insertar serie
    QChart *chart = new QChart();
    chart->legend()->hide();
    chart->addSeries(series);
    chart->createDefaultAxes();
    chart->setTitle("Simple line chart example");
    // Crear widget QChartView (e insertarlo en una ventana):
    QChartView *chartView = new QChartView(chart);
    chartView->setRenderHint(QPainter::Antialiasing);
    MainWindow window;
    window.setCentralWidget(chartView);
    window.show();
    return a.exec();
}

```



7.3 Gráficos de barras y barras horizontales

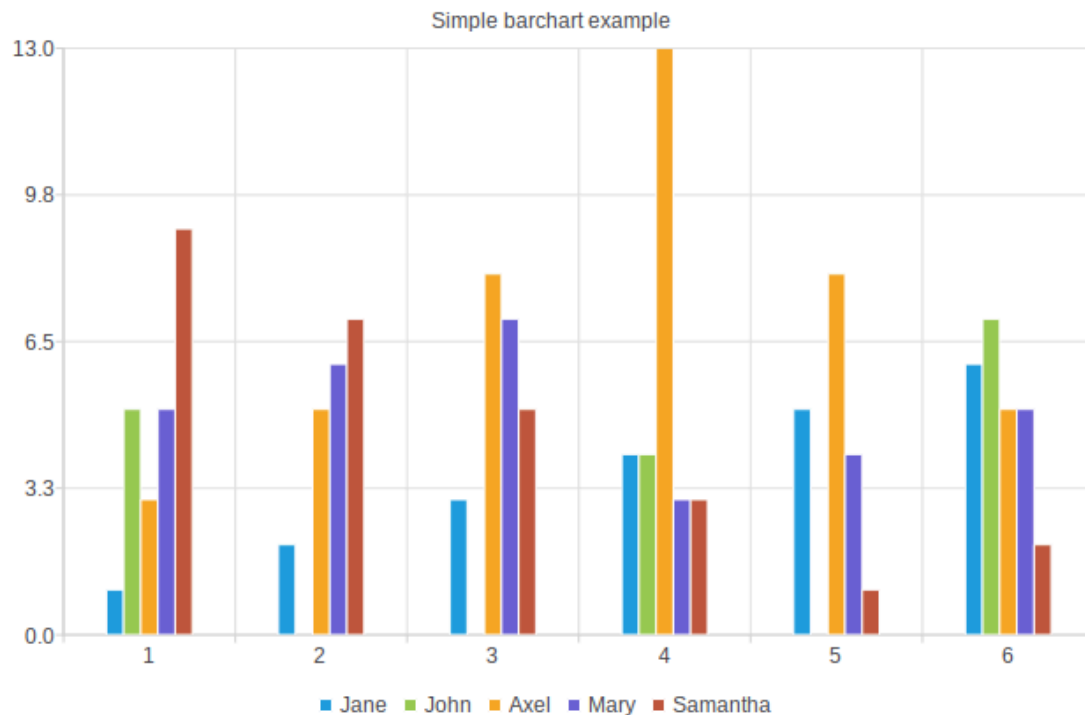
Utilizan conjuntos (`QBarSet`) agrupados en series (`QBarSeries` o `QHorizontalBarSeries`). Son útiles para comparar categorías o grupos.

Barras verticales:

```
#include "mainwindow.h"
#include <QApplication>

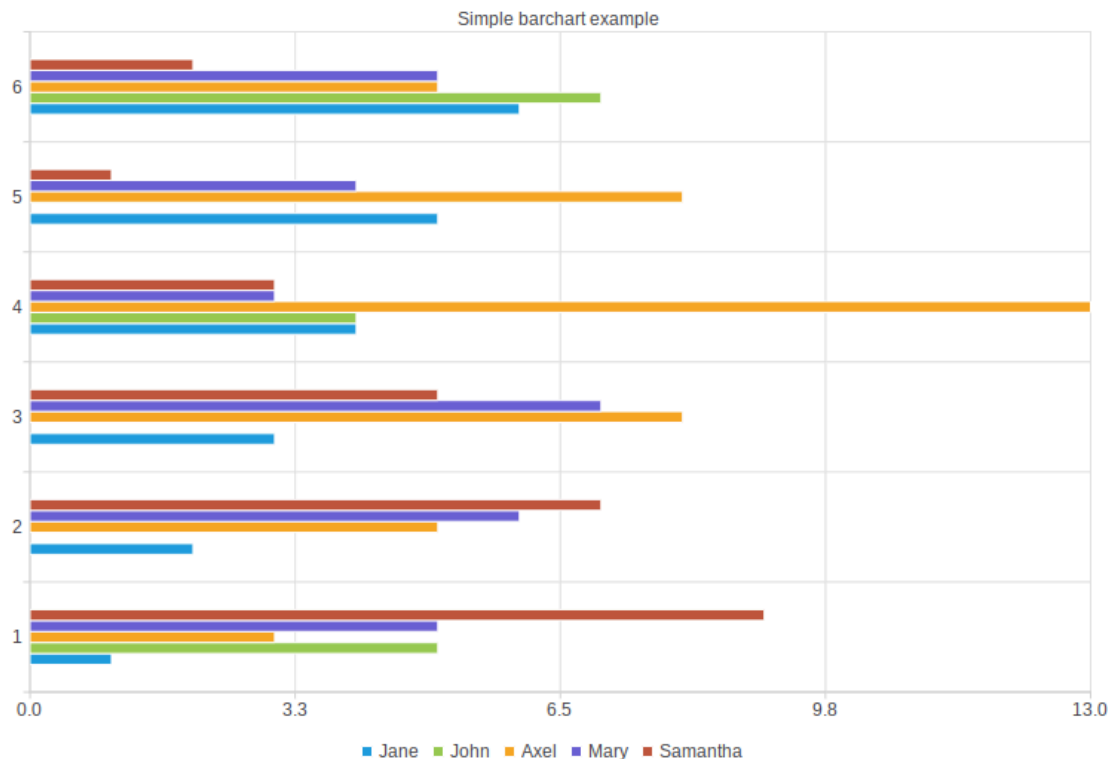
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Crear conjuntos
    QBarSet *set0 = new QBarSet("Jane");
    QBarSet *set1 = new QBarSet("John");
    QBarSet *set2 = new QBarSet("Axel");
    QBarSet *set3 = new QBarSet("Mary");
    QBarSet *set4 = new QBarSet("Samantha");
    *set0 << 1 << 2 << 3 << 4 << 5 << 6;
    *set1 << 5 << 0 << 0 << 4 << 0 << 7;
    *set2 << 3 << 5 << 8 << 13 << 8 << 5;
    *set3 << 5 << 6 << 7 << 3 << 4 << 5;
    *set4 << 9 << 7 << 5 << 3 << 1 << 2;
    // Crear series
    QBarSeries *series = new QBarSeries;
    series->append(set0);
    series->append(set1);
    series->append(set2);
    series->append(set3);
    series->append(set4);

    // Crear gráfico e insertar serie
    QChart *chart = new QChart();
    chart->legend()->setVisible(true);
    chart->legend()->setAlignment(Qt::AlignBottom);
    chart->addSeries(series);
    chart->createDefaultAxes();
    chart->setTitle("Simple line chart example");
    // Crear widget QChartView (e insertarlo en una ventana):
    QChartView *chartView = new QChartView(chart);
    chartView->setRenderHint(QPainter::Antialiasing);
    MainWindow window;
    window.setCentralWidget(chartView);
    window.show();
    return a.exec();
}
```



Barras horizontales: Idéntico a gráfico de barras verticales, con las siguientes modificaciones

```
QHorizontalBarSeries *series = new QHorizontalBarSeries();
series->append(set0);
series->append(set1);
series->append(set2);
series->append(set3);
series->append(set4);
```



7.4 Gráficos de área

Definidos por dos series de línea que delimitan la región a sombrear. Se construyen con `QAreaSeries`.

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    // Se utilizan dos QLineSeries que definen los límites superior e inferior
    QLineSeries *series0 = new QLineSeries();
    QLineSeries *series1 = new QLineSeries();
    *series0 << QPointF(1, 5) << QPointF(3, 7) << QPointF(7, 6) <<
    QPointF(9, 7)
    << QPointF(12, 6) << QPointF(16, 7) << QPointF(18, 5);
    *series1 << QPointF(1, 3) << QPointF(3, 4) << QPointF(7, 3) <<
    QPointF(8, 2)
    << QPointF(12, 3) << QPointF(16, 4) << QPointF(18, 3);
    QAreaSeries *series = new QAreaSeries(series0, series1);

    // Crear gráfico e insertar serie
    QChart *chart = new QChart();
```

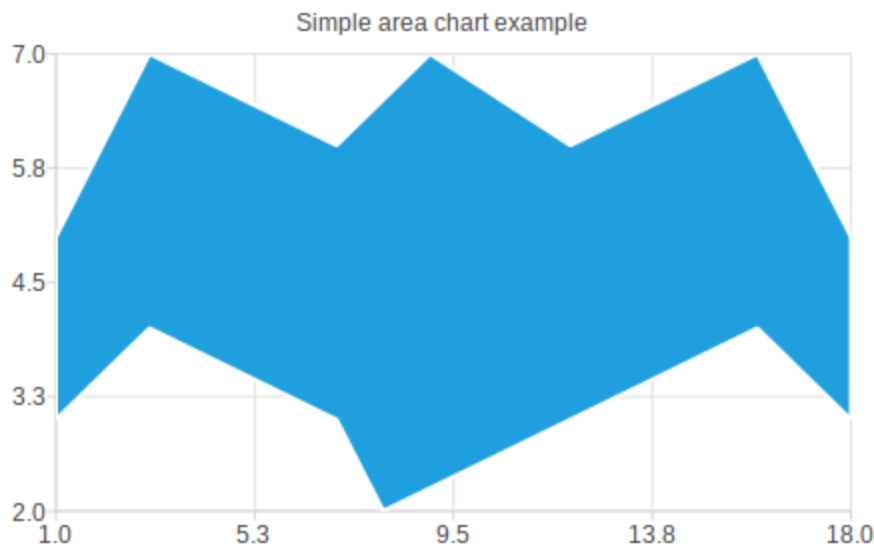
```

chart->legend()->hide();
chart->addSeries(series);
chart->createDefaultAxes();
chart->setTitle("Simple area chart example");

QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing);

MainWindow window;
window.setCentralWidget(chartView);
window.show();
return a.exec();
}

```



7.5 Gráficos de torta

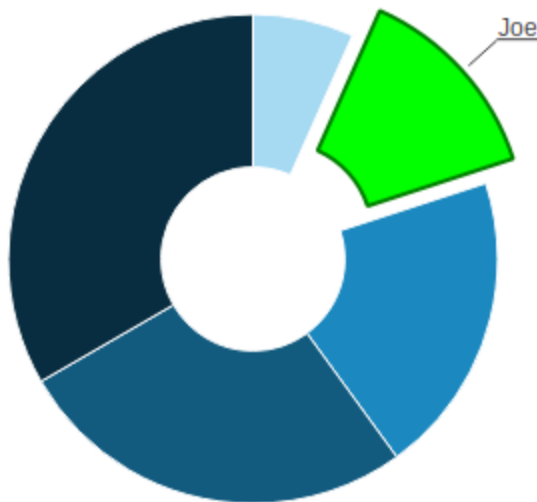
Representan proporciones de un total. La clase `QPieSeries` permite definir sectores y personalizar colores o cortes.

```

QPieSeries *series = new QPieSeries();
series->append("A", 3);
series->append("B", 5);
series->append("C", 2);
series->setHoleSize(0.35);

```

Simple pie chart example



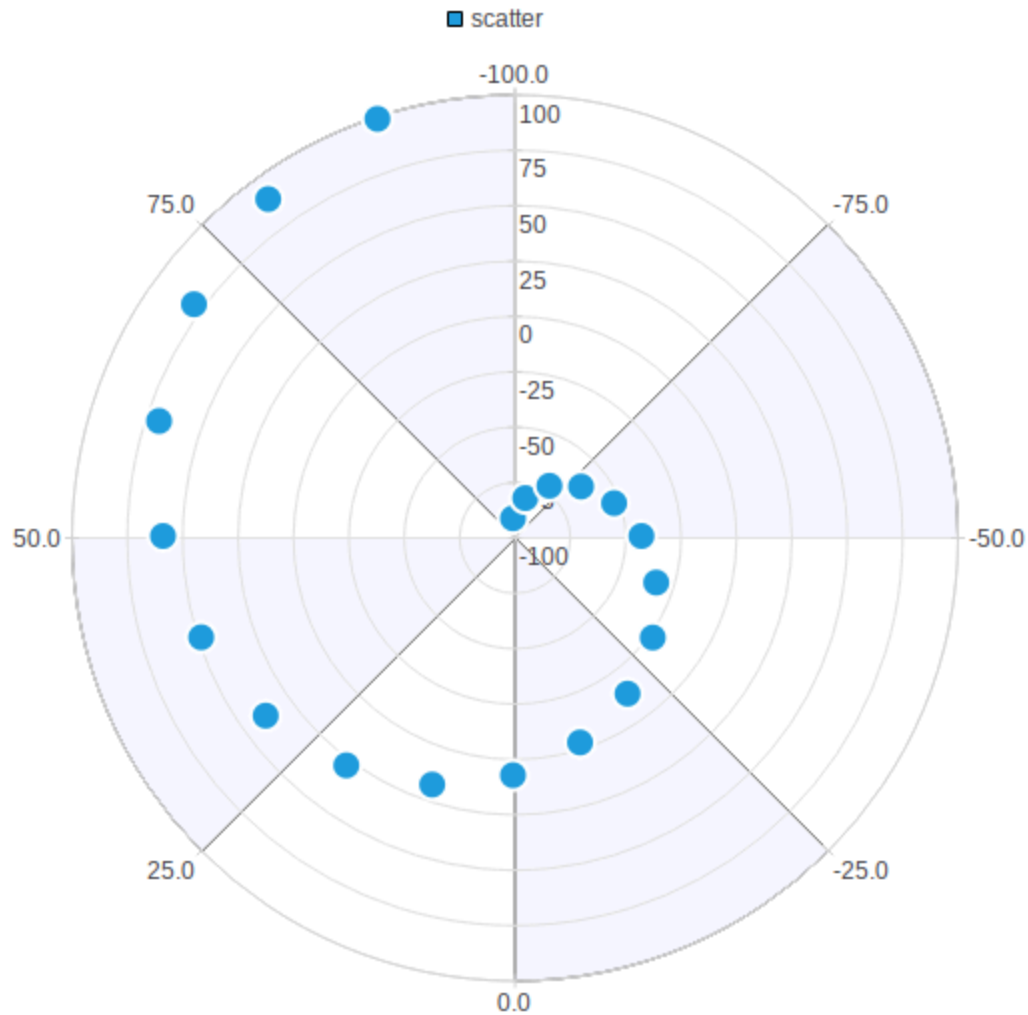
7.6 Gráficos polares

Se basan en ángulos y radios, útiles para mostrar relaciones circulares o datos periódicos. Usan la clase `QPolarChart` y pueden combinarse con `QScatterSeries` o `QLineSeries`.

```
#include "mainwindow.h"
#include <QApplication>
```

```
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    // Variables que almacenan los ángulos mínimo y máximo del gráfico
    const qreal angularMin = -100;
    const qreal angularMax = 100;
    // Variables que almacenan los valores de radio mínimos y máximo
    const qreal radialMin = -100;
    const qreal radialMax = 100;
    // Creación de serie de datos para graficar
    QScatterSeries *series1 = new QScatterSeries();
    series1->setName("scatter");
    for (int i = angularMin; i <= angularMax; i += 10)
        series1->append(i, (i / radialMax) * radialMax + 8.0);
    QPolarChart *chart = new QPolarChart();
    chart->addSeries(series1);
    // Creación de eje angular
    QValueAxis *angularAxis = new QValueAxis();
    // Configuración visual de eje angular
    angularAxis->setTickCount(9);
    angularAxis->setLabelFormat("%.1f");
    angularAxis->setShadesVisible(true);
```

```
angularAxis->setShadesBrush(QBrush(QColor(249, 249, 255)));  
// Conexión de eje angular con gráfico  
chart->addAxis(angularAxis, QPolarChart::PolarOrientationAngular);  
  
// Creación de eje radial  
QValueAxis *radialAxis = new QValueAxis();  
//Configuración de eje radial  
radialAxis->setTickCount(9);  
radialAxis->setLabelFormat("%d");  
// Conexión de eje radial con gráfico  
chart->addAxis(radialAxis, QPolarChart::PolarOrientationRadial);  
// Conexión de ejes con serie  
series1->attachAxis(radialAxis);  
series1->attachAxis(angularAxis);  
// Rango de ejes  
radialAxis->setRange(radialMin, radialMax);  
angularAxis->setRange(angularMin, angularMax);  
  
QChartView *chartView = new QChartView();  
chartView->setChart(chart);  
chartView->setRenderHint(QPainter::Antialiasing);  
  
MainWindow window;  
window.setCentralWidget(chartView);  
window.resize(800, 600);  
window.show();  
return a.exec();  
}
```

7.7 Personalización y guardado

Los gráficos pueden personalizarse con títulos, leyendas, colores o animaciones:

```
chart->legend()->setVisible(true);
chart->setAnimationOptions(QChart::AllAnimations);
```

También pueden exportarse como imágenes:

```
QPixmap p = chartView->grab();
p.save("grafico.png", "PNG");
```

Con **Qt Charts**, es posible integrar visualizaciones directamente en las aplicaciones C++ sin depender de bibliotecas externas, facilitando la creación de paneles e interfaces interactivas.